Trabajo de Fin de Grado

Grado en Ingeniería Informática

Priorización de Objetivos en el Instrumento Telescópico MIRADAS

Prioritization of Objectives in the MIRADAS Telescope Instrument

Jorge Elías García

La Laguna, 22 de mayo de 2025

D. **Jorge Riera Ledesma**, profesor Titular de Universidad adscrito al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, como tutor

C E R T I F I C A

Que la presente memoria titulada:

"Priorización de Objetivos en el Instrumento Telescópico MIRADAS."

ha sido realizada bajo su dirección por D. **Jorge Elías García**.

Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos, firman la presente en La Laguna a 22 de mayo de 2025

Agradecimientos

Jorge Riera Ledesma, tutor académico.

Licencia

© Esta obra está bajo una licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional.

Resumen

El instrumento telescópico MIRADAS está diseñado para el análisis espectrográfico de objetos celestes mediante varios brazos robóticos, cada uno equipado con un sensor. Esta arquitectura permite la observación simultánea de múltiples objetivos, aunque algunos requieren el uso combinado de varios sensores para alcanzar la resolución deseada. Dado el elevado coste del tiempo de observación, este trabajo se centra en el desarrollo de un algoritmo que optimice el uso del instrumento.

El problema se ha modelado como una variante del *Consistent Traveling Salesman Problem* (CTSP), una abstracción que refleja las restricciones operativas de MIRADAS y vincula la planificación de observaciones con la ejecución eficiente de operaciones sincronizadas. En concreto, se aborda una versión del CTSP en la que no se permiten tiempos de espera entre operaciones, lo que incrementa significativamente la complejidad del problema.

El trabajo propone un algoritmo heurístico para resolver esta variante. Su eficacia ha sido validada mediante 756 instancias adaptadas de la conocida librería TSPLIB, con hasta 500 operaciones por caso. Esta experimentación ha permitido evaluar la calidad de las soluciones generadas por el algoritmo propuesto.

Palabras clave: Optimización, Consistent Traveling Salesman Problem, Heurística, Factibilidad, Enrutamiento, Plantilla de precedencia.

Abstract

The MIRADAS telescopic instrument is designed for the spectrographic analysis of celestial objects using multiple robotic arms, each equipped with a sensor. This architecture enables the simultaneous observation of multiple targets, although some require the combined use of several sensors to achieve the desired resolution. Given the high cost of observation time, this work focuses on developing an algorithm that optimizes the use of the instrument.

The problem has been modeled as a variant of the Consistent Traveling Salesperson Problem (CTSP), an abstraction that captures MIRADAS' operational constraints and links observation planning with the efficient execution of synchronized operations. Specifically, this study addresses a version of the CTSP in which waiting times between operations are not allowed, significantly increasing the problem's complexity.

The work proposes a heuristic algorithm to solve this variant. Its effectiveness has been validated through 756 adapted instances from the well-known TSPLIB library, with up to 500 operations per case. This experimentation has allowed for an evaluation of the quality of the solutions generated by the proposed algorithm.

Keywords: Optimization, Consistent Traveling Salesman Problem, Heuristic, Feasibility, Routing, Precedence template.

Índice

1. Introducción **1**
1.1. Descripción de MIRADAS . . . 1
1.2. Antecedentes . . . 1
1.3. Estado del Arte . . . 2
1.4. Objetivos . . . 3
1.5. Fases y Desarrollo del Proyecto . . . 3

2. Consistent Traveling Salesman Problem **4**
2.1. Planteamiento del problema . . . 4
2.2. Parámetros de entrada . . . 4
2.3. Definición de una solución factible . . . 5
2.4. Ejemplos de soluciones factibles . . . 6
2.5. Correspondencia con MIRADAS . . . 11

3. Formulación matemática **12**
3.1. Restricciones comunes . . . 12
3.1.1. Caminos hamiltonianos . . . 12
3.1.2. Restricciones de precedencia . . . 13
3.1.3. Limitación de los tiempos de servicio . . . 13
3.2. Restricciones específicas . . . 13
3.2.1. Consistent Traveling Salesman problem (CTSP) sin tiempos de espera . 13
3.2.2. CTSP con tiempos de espera . . . 14
3.3. Modelos matemáticos . . . 14

4. Instancia a Resolver **15**
4.1. Edge Weight Type . . . 16
4.2. Demand Section . . . 16

5. Solución **17**
5.1. **Descripción General del Algoritmo** . . . 17
5.2. **Solución TSP** . . . 17
5.2.1. 2-opt (First Improvement) . . . 18
5.3. **Solución CTSP1** . . . 20
5.3.1. **2-opt Generalizado** . . . 20
5.4. **Pseudocódigo** . . . 21
5.4.1. **Devolver Solución** . . . 21
5.4.2. **Construir Solución** . . . 21
5.5. **Verificador** . . . 23

6. Visualización Resultados Obtenidos **24**
6.1. Análisis por Familia de Instancias . . . 24
6.1.1. Complejidad Familias Ft . . . 26
6.2. Análisis por Número de Clientes . . . 28
6.3. Análisis por Número de Tareas . . . 31
6.3.1. Impacto Ventana Temporal Máxima . . . 32
6.4. Análisis por Número de Días de Servicio . . . 34
6.5. Análisis Global . . . 35

6.6. Justificación Número Iteraciones 36

7. Conclusiones y Líneas Futuras **38**

8. Summary and Conclusions **39**

9. Presupuesto **40**

10. Bibliografía y Referencias **41**

Índice de figuras

1. Representación gráfcia de una solución factible para el CTSP1 a partir de la entrada burma14_p3_f50_lH. 9
2. Representación gráfica de una solución factible para el CTSP2 a partir de la entrada burma14_p3_f50_lH. 10
3. Ejemplo de instancia CTSP 15
4. Ruta inicial TSP 18
5. Ruta TSP tras reconexión con 2-opt 19
6. Reresentación gráfica gap frente al número de clientes 29
7. Reresentación gráfica tiempo ejecución frente al número de clientes 30
8. Reresentación gráfica probabilidad encontrar solución factible frente al número de clientes 30
9. Reresentación gráfica gap frente a ventanta temporal máxima 33

Índice de cuadros

1. Demandas de cada localización en el conjunto L para cada día de servicio en K, para la entrada burma14_p3_f50_lH. 7
2. Descripción general de los campos de la instancia CTSP 16
3. Resultados del algoritmo, agrupando por familias de instancias. 25
4. Rango de Gaps y Moda Soluciones Encontradas. 26
5. Resultados del algoritmo, agrupando por número de clientes. 28
6. Resultados del algoritmo, agrupando por número de tareas. 31
7. Rango de tareas para las familias ft. 32
8. Impacto ventanta temporal máxima. 33
9. Resultados del algoritmo, agrupando por días de servicio y número de tareas. . . 34
10. Resultados medios. 35
11. Resultados medios con CTSP métrico. 35
12. Resultados medios CTSP no métrico. 35
13. Resultados algoritmo, primeras 473 instancias 36
14. Resultados algoritmo modificando número de iteraciones, primeras 473 instancias 36
15. Presupuesto de Equipos y Licencias 40
16. Coste de Mano de Obra 40
17. Coste Total del Proyecto 40

Índice de algoritmos

1. Build Solution 21
2. Solve From TSP 22

1. Introducción

1.1. Descripción de MIRADAS

En la Astrofísica moderna, muchos de los desafíos científicos más importantes sólo pueden abordarse de manera eficiente utilizando instrumentos de alta resolución espectral, montados en telescopios con una gran área de recolección de luz. Este tipo de instrumentos permite analizar fenómenos astronómicos con gran precisión y profundidad, respondiendo así a cuestiones fundamentales sobre el universo. En los últimos años, la demanda de espectrógrafos multiobjeto (MOS) ha crecido significativamente debido a su capacidad para observar múltiples objetivos de manera simultánea, característica que resulta crucial en investigaciones donde se necesita recolectar datos de varias fuentes de luz en un mismo campo de visión.

En este contexto, el **Espectrógrafo Astronómico de Infrarrojo Medio de Resolución Moderada (MIRADAS**), se presenta como un MOS de infrarrojo cercano diseñado específicamente para el Gran Telescopio Canarias (GTC). Este instrumento cuenta con un sistema de multiplexación (MXS) compuesto por 12 Unidades de Campo Integral (IFU) independientes y desplegables. Cada IFU utiliza un brazo de sonda con un espejo de captura que permite al instrumento observar hasta 12 objetivos distintos en un mismo campo de visión, según las necesidades del usuario.

MIRADAS es un proyecto de carácter científico, desarrollado por un consorcio internacional que incluye la Universidad de Florida, la Universitat de Barcelona, la Universidad Complutense de Madrid, el Instituto de Astrofísica de Canarias y el Institut d'Estudis Espacials de Catalunya. Al ser un instrumento común en el GTC, MIRADAS deberá adaptarse a múltiples configuraciones de observación, según las necesidades científicas específicas de cada proyecto. Sin embargo, para maximizar la eficiencia en cada observación y proteger el hardware de los brazos de sonda, el instrumento debe contar con un sistema de configuración óptimo.

El software de control del MXS es el encargado de gestionar el movimiento de cada brazo de sonda. Para ello, realiza dos pasos secuenciales: en el primero, se asigna cada objetivo a un solo brazo de sonda, según su posición en el campo de visión. En el segundo paso, deberá encontrarse la ruta para cada uno de los brazos, de tal manera que todos ellos visiten todos los objetivos que les han sido asignados. Se deberá tener en cuenta que, si un objetivo tiene que ser visitado por más de un brazo, la diferencia de tiempo desde la visita del primer brazo hasta la visita del último no puede superar un tiempo máximo, el cual será definido por el usuario.

1.2. Antecedentes

Dado un sistema MXS con varios brazos de sonda y un conjunto de puntos objetivo en su plano focal, el problema de la determinación de trayectorias se puede descomponer en dos subproblemas diferentes y más manejables. El primero de ellos consiste en asignar los brazos de sonda del sistema a los objetivos definidos por el usuario, según un criterio específico. Esta tarea, denominada asignación de objetivos a brazos, se formula como un problema de optimización combinatoria (COP, por sus siglas en inglés). En el contexto de las matemáticas aplicadas y las ciencias de la computación teórica, la optimización combinatoria es un campo activo que combina técnicas de combinatoria y programación lineal con la teoría de algoritmos, con el objetivo de resolver problemas de optimización en estructuras discretas. Esta área también se relaciona estrechamente con la investigación operativa, la teoría de algoritmos y la teoría de la complejidad computacional,

todos ellos de gran relevancia en Ingeniería Informática.

Una vez completada la asignación, el siguiente paso es calcular trayectorias libres de colisiones para cada brazo. Estas trayectorias consisten en el recorrido geométrico que cada brazo debe seguir para alcanzar su objetivo asignado y en un perfil temporal que especifica la evolución de su movimiento. Ambos aspectos se analizan en profundidad en la sección de planificación de movimiento. El proceso de establecer una secuencia óptima de movimiento para los brazos de sonda, que permita satisfacer todos los objetivos definidos, se puede describir matemáticamente a través del **Consistency Traveling Salesman Problem (CTSP),** una variación del problema clásico del viajante de comercio.

1.3. Estado del Arte

Si el conjunto de objetivos supera en tamaño al conjunto de brazos de sonda, será necesario realizar múltiples observaciones para capturar todos los objetivos requeridos. Incluso cuando ambos conjuntos tienen la misma cardinalidad, podría requerirse más de una observación, especialmente si el algoritmo detecta incompatibilidades en la configuración. En este contexto, el algoritmo de asignación de objetivos a brazos realiza ejecuciones sucesivas, determinando en cada ocasión el conjunto de objetivos que debe observarse y la configuración necesaria de los brazos de sonda para llevar a cabo esta tarea. En la primera iteración, el algoritmo selecciona objetivos del conjunto total, mientras que en las iteraciones siguientes reduce el conjunto de opciones, eliminando aquellos objetivos ya asignados en ejecuciones anteriores.

En el problema estándar del viajante de comercio, se requiere encontrar la ruta más corta que permite a un viajante visitar una serie de ciudades, pasando solo una vez por cada una y regresando al punto de origen. El **CTSP**, sin embargo, añade un conjunto de restricciones de consistencia que obligan a que ciertas condiciones se mantengan durante el recorrido. En el contexto de los brazos de sonda, el CTSP se adapta para incluir restricciones específicas de cada brazo, como la necesidad de evitar colisiones y de optimizar el orden de los objetivos de manera que minimice el tiempo y maximice la eficiencia de observación. En este problema, cada brazo de sonda actúa como un "viajante" que debe alcanzar varios puntos objetivos dentro del campo de visión del instrumento. A diferencia del problema clásico, aquí se busca optimizar no solo el recorrido total, sino también la secuencia de movimientos, respetando al mismo tiempo las limitaciones mecánicas y las posibles interacciones entre los brazos. Esto implica formular un modelo de optimización combinatoria capaz de gestionar estas condiciones adicionales, permitiendo que el sistema asigne a cada brazo una secuencia de objetivos y una trayectoria libre de colisiones.

1.4. Objetivos

En primer lugar, la comprensión y redacción formal del problema presentado, respetando y clarificando las analogías entre MIRADAS y CTSP. Consecuentemente, añadir una descripción detallada del mismo, ejemplos de instancias a resolver y soluciones factibles, entre otros elementos de interés para la comprensión del problema.

En segundo lugar, la elaboración de un *parser* para la lectura de una instancia CTSP a un objeto *instance* en C++, con el fin de su posterior procesamiento. De esta manera, se podrá validar el algoritmo con más de 756 instancias adaptadas de la reconocida librería TSPLIB.

En tercer lugar, la **implementación de un algoritmo aproximado para resolver el CTSP, atendiendo a las restricciones adicionales del problema**. Asimismo, la elaboración de un verificador que nos permita, dada una instancia y una posible solución, comprobar si se trata de una solución factible o no.

En último lugar, la visualización gráfica de los resultados obtenidos, para de este modo reflejar visualmente el comportamiento del algoritmo codificado y comprobar la calidad del mismo.

1.5. Fases y Desarrollo del Proyecto

1. Comprensión y redacción formal del problema.
2. Elaboración de un *parser* para la lectura de una instancia CTSP a un objeto instance en C++.
3. Planteamiento y codificación de un algoritmo aproximado.
4. Implementación de optimizaciones.
5. *Testing* del algoritmo y visualización de resultados.

2. Consistent Traveling Salesman Problem

Una vez comprendido el problema MIRADAS, entendemos que se trata de un problema de optimización combinatoria relacionado con la asignación de rutas. Nos encontramos ante una variante del Consistent Traveling Salesman Problem, donde no se permiten tiempos de espera. A esta variante la denominaremos CTSP1, y se trata de una abstracción que puede modelizar el movimiento de los brazos del instrumento astronómico MIRADAS para su uso eficiente en la investigación astronómica. **Si es difícil encontrar una solución factible para CTSP, aún permitiendo tiempos de espera, lo será más para CTSP1.**

A continuación, se explica detalladamente el CTSP (así como sus variantes), ya que, en definitiva, es nuestro problema a resolver.

2.1. Planteamiento del problema

El problema del Consistent Traveling Salesman Problem (CTSP) surge en contextos donde es necesario visitar a clientes de manera regular y mantener la coherencia en las rutas planificadas. Ejemplos típicos incluyen la distribución periódica de mercancías o servicios de mantenimiento programados.

A continuación introducimos la notación utilizada para describir una instancia del CTSP, lo que nos ayudará a comprender posteriormente este problema de optimización. Definiremos formalmente los elementos que forman parte de los datos de entrada del problema, como clientes, operaciones de entrega y sus parámetros asociados, y posteriormente estableceremos los elementos de una solución de este problema.

2.2. Parámetros de entrada

Considérese un conjunto $L = \{1, \dots, n\}$, conteniendo n localizaciones de clientes. Un vehículo de reparto debe visitar periódicamente estos clientes. Dicho vehículo comienza su jornada en depósito de partida, designado por 0 y debe concluirla en el depósito de llegada $n+1$. Denotaremos por L^+ al conjunto L extendido con los depósitos de partida y llegada, es decir, $L^+ = L \cup \{0, n+1\}$.

Los días de reparto vienen dados por un conjunto K de *días de servicio*, asumiendo que $|K| \geq 2$.

Un conjunto $O \subset L \times K$ de *operaciones de entrega* determina aquellas localizaciones que serán visitadas por el vehículo cada día de servicio. Cada elemento del conjunto O representa una operación de entrega designada por un par (l, k), para $l \in L$ y $k \in K$. Dicha operación establece que el vehículo debe visitar la localización l durante el día de servicio k. Designaremos O^+ al conjunto O extendido con las operaciones de partida y llegada, es decir, $O^+ = O \cup \{(0, k), (n+1, k) \mid k \in K\}$.

Denotaremos por t_{ij} el tiempo requerido para efectuar el desplazamiento entre las dos localizaciones asociadas a las operaciones i y j, para $i, j \in O^+$. Además, efectuar dicho desplazamiento produce en un coste c_{ij}.

Por último, la consistencia en el CTSP viene especificada por un valor w. Este valor w representa la amplitud máxima de la ventana temporal en la que se debe planificar el servicio a cada cliente

cada día de servicio. Es decir, si se planifica una operación de entrega $i = (l, k)$ en el instante $\tilde{s}_i$, entonces cualquier otra operación de entrega $j = (l, k')$ que involucre al mismo cliente l otro día de servicio k', y que esté planificada en el instante $\tilde{s}_j$, debe cumplir $|\tilde{s}_i - \tilde{s}_j| \leq w$, para $i = (l, k), j = (l, k') \in O$.

Resumiendo, una entrada del CTSP consiste en un conjunto L^+ de localizaciones de clientes y depósitos, un conjunto K de días de servicio, un conjunto O^+ de operaciones, los vectores t y c de tiempo y coste, y la amplitud máxima w de la ventana de entrega.

2.3. Definición de una solución factible

Una solución factible del CTSP consiste en la planificación de las operaciones de entrega en los días de servicio de tal forma que se respeten las restricciones temporales impuestas por la amplitud máxima de la ventana temporal a la que está sujeta la planificación de las operaciones de entrega asociadas a un mismo clientes. Esta sección define formalmente una solución factible.

Con el fin de modelizar el conjunto de soluciones factibles, a continuación definiremos dos familias de grafos dirigidos: $G(k)$, para $k \in K$, y $H(l)$, para $l \in L$. La primera familia nos permitirá definir las rutas que el vehículo debe seguir en cada día de servicio, mientras que la segunda familia nos permitirá establecer las relaciones temporales entre las operaciones de entrega asociadas a cada cliente.

Sea $G(k) = (V(k), A(k))$, para cada $k \in K$, una familia de grafos dirigidos representando todas las posibles rutas que puede efectuar el vehículo durante cada día de servicio k. Denotemos por $O(k) \subset O$, el subconjunto de operaciones de entrega previstas para el día de servicio k, para cada $k \in K$. Entonces, definimos el conjunto de vértices $V(k)$ como $V(k) = \{O(k) \cup \{(0, k), (n + 1, k)\}\}$. Es decir, $V(k)$ es el conjunto de operaciones establecidas para el día de servicio k. El conjunto de arcos $A(k)$ contiene las conexiones desde $(0, k)$ hasta las operaciones de entrega $O(k)$, las conexiones entre cada par de estas operaciones de entrega $O(k)$, y las conexiones entre los elementos de $O(k)$ y $(n + 1, k)$.

Sea $H(l) = (W(l), B(l))$, para cada $l \in L$, una familia de grafos completos dirigidos representando relaciones entre las operaciones asociadas a un mismo cliente l. Sea su conjunto de vértices $W(l) \subset O$, el subconjunto de operaciones de entrega asociadas a un mismo cliente l. Su conjunto de arcos $B(l)$ contiene todas las conexiones entre los elementos de $W(l)$, para cada $l \in L$.

Una solución factible para el CTSP contiene dos componentes:

1. Un camino hamiltoniano $P(k) \subseteq A(k)$ en $G(k)$, para cada $k \in K$, comenzando cada uno en la operación de partida $(0, k)$, visitando a todas las operaciones de entrega $O(k)$ y concluyendo en la operación de llegada $(n + 1, k)$.

2. Los valores $\tilde{s}$ de un vector de planificación indexado en el conjunto de operaciones O^+, que asigna a cada operación $i \in O^+$ un instante de tiempo $\bar{s}_i$. Este vector $\tilde{s}$ es el resultado de la planificación de las operaciones O^+ a partir de los tiempos t_{ij} asociados a los arcos (i, j) de cada camino hamiltonianos $P(k)$, para cada $k \in K$. Los valores de $\tilde{s}$ deben satisfacer las siguientes restricciones:

$$\tilde{s}_j - \tilde{s}_i \leq w \qquad (i, j) \in B(l), l \in L. \tag{1}$$

Es decir, la diferencia entre los tiempos de servicio de dos operaciones de entrega asociadas al mismo cliente $l \in L$ no puede exceder la amplitud w de la ventana temporal.

Nótese que en la condición (1) no considera el valor absoluto de la diferencia entre $\tilde{s}_j$ y $\tilde{s}_i$. Esto se debe a que los arcos en $B(l)$ incluyen todas las posibles conexiones entre las operaciones de entrega asociadas al cliente l, para cada cliente $l \in L$. Así, siempre habrá algún par (i, j) tal que $\tilde{s}_i \leq \tilde{s}_j$.

El CTSP admite dos variaciones de acuerdo a la manera en la que se cree el vector de planificación $\tilde{s}$. La primera variación, denominada en la literatura CTSP1, fuerza que tan pronto el vehículo llegue a la localización de destino se efectúe el servicio. Por tanto no considera tiempos de espera. La segunda variación, conocida como CTSP2, permite que le vehículo efectúe tiempos de espera. De esta manera, algunas pausas pueden forzar que la planificación de ciertas de las tareas se ajuste, evitando la violación de las restricciones (1).

2.4. Ejemplos de soluciones factibles

Esta sección presenta un ejemplo numérico de solución factible para cada una de las dos variaciones del CTSP. Para ambos casos consideramos la entrada de prueba $\mathrm{burma14_p3_f50_lH}$, que pertenece al conjunto de entradas propuestas por Subramanyam and Gounaris en sus trabajos [1, 2] sobre ambas variaciones del problema. Estas instancias de prueba están disponibles en el sitio http://gounaris.cheme.cmu.edu/datasets/contsp/.

En concreto, esta entrada cuenta con un conjunto con 14 localizaciones $L = \{1, 2, \ldots, 14\}$. De éstas, las localizaciones $\{2, \ldots, 14\}$ corresponden a clientes. La entrada considera 3 días de servicio, es decir $K = \{1, 2, 3\}$ y una amplitud máxima de la ventana temporal $w = 617$. Para el caso concreto del CTSP2, se ha establecido un tiempo máximo de servicio $T = 2817$.

Las demandas de cada localización en el conjunto L para cada día de servicio en K se muestran en la tabla 1. Se observa que en esta entrada en particular las localizaciones 2, 10 y 11 no presentan ninguna demanda en ningún día de servicio. La Localización 1, por otro lado, presenta demanda todos los días, ya que se trata del depósito. Por otro lado, el cliente asociado a la localización 5 requiere ser visitado cada uno de los tres días de servicio, mientras que los clientes asociados a las localizaciones 8 y 13 demandan servicio un único día. Esta tabla de demandas indica que la entrada contiene 19 operaciones de entrega en total, además de 3 operaciones de partida y 3 operaciones de llegada.

	L													
K	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	✓		✓	✓	✓	✓	✓		✓			✓		
2	✓		✓		✓			✓	✓			✓		✓
3	✓			✓	✓	✓	✓						✓	✓

Cuadro 1: Demandas de cada localización en el conjunto L para cada día de servicio en K, para la entrada burma14_p3_f50_lH.

Las figuras 1 y 2 muestran gráficamente soluciones factibles obtenidas para el CTSP1 y el CTSP2, respectivamente. Ambas figuras contienen dos subfiguras que muestran la planificación temporal de las operaciones de entrega para cada uno de los 3 días de servicio, y la misma planificación temporal de las operaciones de entrega para cada una de las localizaciones asociadas a los clientes, respectivamente.

El instante de planificación de una operación viene representado por un segmento vertical con una leyenda en su parte inferior que indica la denominación de dicha operación. El número situado a la derecha de cada fila indica el día de servicio en las subfiguras 1(a) y 2(a). Así, por ejemplo, el primer segmento vertical situado en la fila 1 de la subfigura 1(a) indica que la operación de entrega $(3, 1)$, asociada a la localización 3 y al día de servicio 1, se ha planificado en el instante 510.

Estas dos figuras representan la transición entre operaciones mediante una rectángulo sombreado en gris, en cuyo interior se indica el tiempo de transporte entre las dos operaciones implicadas. En la subfigura 2(a), se puede observar también una zona sombreada en color violeta en la transición que precede a la operación $(12, 1)$ desde la operación $(7, 1)$. Esta zona sombreada indica que el vehículo ha efectuado un tiempo de espera de 146 unidades antes de efectuar la operación de entrega $(12, 1)$, después de haber transitado durante 163 unidades de tiempo desde la operación $(7, 1)$.

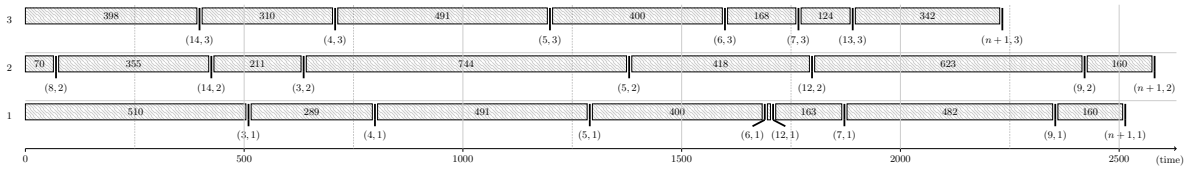
Aunque no aparece reflejado de forma expícita en las figuras 1 y 2, Cada día de servicio comienza con la operación de partida $(0, k)$, para $k \in K$. Todas las planificaciones también concluyen con la operación de llegada $(n + 1, k)$, para $k \in K$.

Las subfiguras 1(b) y 2(b) representan exactamente la misma planificación temporal de las operaciones de entrega, pero organizadas por localizaciones asociadas a los clientes. Estas figuras facilitan la visualización del cumplimiento de las restricciones temporales impuestas por la amplitud máxima de la ventana temporal. En ellas se muestra un rectángulo sombreado en azul cuyo centro se sitúa en la media de los instantes de planificación de las operaciones asociadas a un mismo cliente. La amplitud de este rectángulo sombreado en azul representa la amplitud máxima de la ventana temporal. De forma gráfica podemos verificar que la planificación de las operaciones de entrega asociadas a un mismo cliente cumple con la restricción temporal impuesta por la amplitud máxima de la ventana temporal, al no exceder los límites de este rectángulo. A la derecha de cada fila se muestra el identificador del cliente asociado a las operaciones representadas en dicha fila.

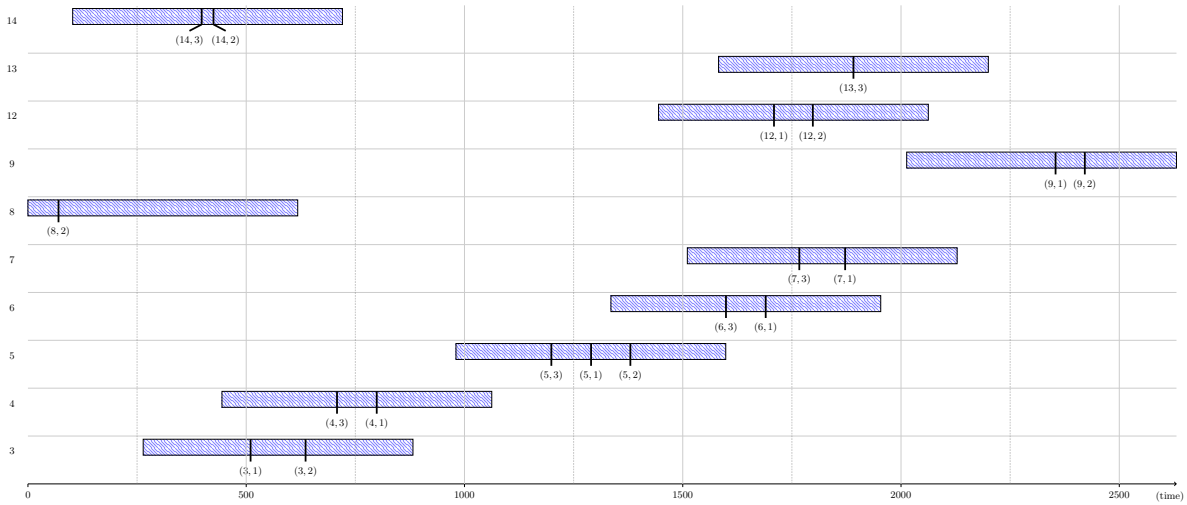
Como indicábamos con anterioridad, la figura 1 corresponde a una solución factible del CTSP1 para la entrada $\mathrm{burma14_p3_f50_lH}$. En esta solución, el vehículo efectúa el servicio a cada cliente tan pronto llega a su localización. El primer día de servicio efectúa sus entregas consumiendo 2514 unidades de tiempo (nótese que el valor de la transición entre las operaciones $(6, 1)$ y $(12, 1)$ es de 19 unidades de tiempo). El segundo día de servicio consume 2581 unidades de tiempo, y el tercer día de servicio consume 2533 unidades de tiempo. Puesto que el tiempo total consumido coincide con el valor del coste total, ya que no existen tiempos de espera, el coste de esta solución es 7328.

Por otro lado, la figura 2 muestra una solución factible del CTSP2 para la misma entrada. En esta solución, el vehículo efectúa un tiempo de espera de 146 unidades antes de alcanzar la operación $(12, 1)$. El primer día de servicio efectúa sus entregas consumiendo 2660 unidades de tiempo, aunque el coste de esta ruta es 2514, puesto que se ha efectuado un tiempo de espera. El segundo día de servicio consume 2561 unidades de tiempo, y el tercer día de servicio consume 2233 unidades de tiempo. Ninguna de las rutas supera el tiempo máximo de servicio $T = 2817$. El coste de esta solución es 7308, 20 unidades menos que en el caso del CTSP1.

Nótese que la inclusión de tiempos de espera facilita encontrar soluciones factibles con respecto a la restricción temporal impuesta por la amplitud máxima de la ventana temporal. Véase el ejemplo de las operaciones $(12, 1)$ y $(12, 2)$ en la subfigura 2(b). Se observa que ambas operaciones ajustan el tiempo de servicio a la amplitud máxima de la ventana temporal, al permitir que el vehículo espere 146 unidades de tiempo el primer día de servicio antes de efectuar la operación $(12, 2)$. En caso contrario, esta solución no sería factible.

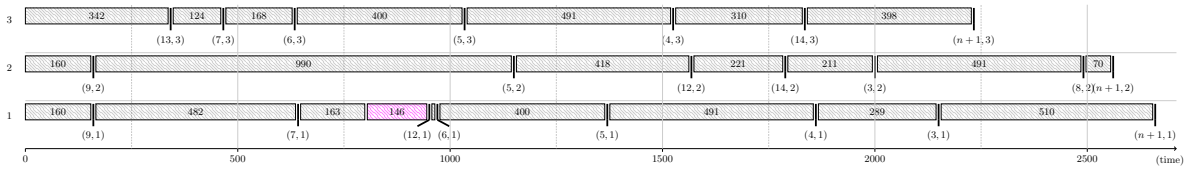


(a) Planificación temporal de las operaciones para cada uno de los 5 días de servicio. Cada fila representa el día de servicio indicado a la derecha.

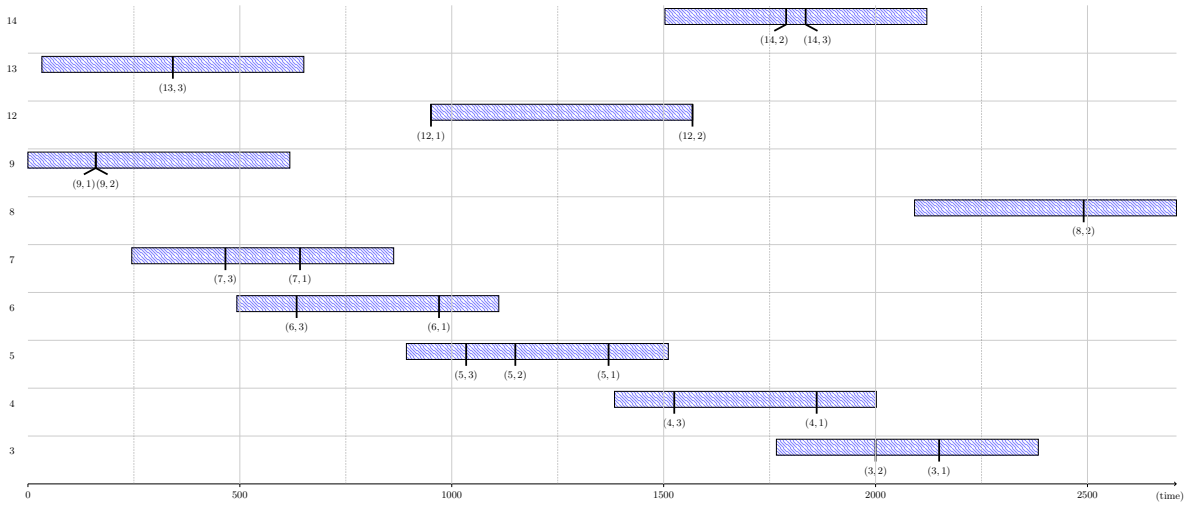


(b) Planificación temporal de las operaciones para cada una de las 13 localizaciones asociadas a los clientes. Cada fila representa la localización indicada a la derecha.

Figura 1: Representación gráfcia de una solución factible para el CTSP1 a partir de la entrada burma14_p3_f50_lH.



(a) Planificación temporal de las operaciones para cada uno de los 5 días de servicio. Cada fila representa el día de servicio indicado a la derecha.



(b) Planificación temporal de las operaciones para cada una de las 13 localizaciones asociadas a los clientes. Cada fila representa la localización indicada a la derecha.

Figura 2: Representación gráfica de una solución factible para el CTSP2 a partir de la entrada burma14_p3_f50_lH.

2.5. Correspondencia con MIRADAS

La definición previa del CTSP1 puede aplicarse para nuestro problema. Sin embargo, debemos entender la analogía entre los términos utilizados para describir el problema y los términos que aparecen en la definición de MIRADAS.

En primer lugar, los días de servicio corresponden a los brazos robóticos. Consecuentemente, en el ejemplo anterior de solución factible, encontramos 3 brazos robóticos que deberán dar servicio a diferentes clientes.

En segundo lugar, cuando se habla de clientes, hace referencia a los objetivos a observar por parte de los brazos. Además, en el ejemplo de solución factible, el cliente 1 se asocia al depósito del que parten y donde deben finalizar los brazos robóticos, por lo que debe ser visitado por todos ellos.

En tercer lugar, una operación dentro del problema de MIRADAS corresponde a que un brazo visite un objetivo específico, en un instante de tiempo determinado. Asimismo, debido a que los tiempos de visita son constantes, simplificamos el modelo despreciando dichos tiempos. Esto se debe a que no modificarán el resultado de la optimización, pues las rutas encontradas serán las mismas.

En definitiva, si un cliente debe recibir servicio en varios días, se traduce a que un objetivo debe ser visitado por varios brazos robóticos. Por lo tanto, cuando se requiere que un cliente reciba servicio en varios días respetando una ventana temporal máxima, significa que un objetivo debe ser visitado por varios brazos robóticos, asegurándose de que el tiempo transcurrido entre la primera y la última visita no supere el límite establecido.

3. Formulación matemática

En esta sección caracterizamos formalmente el conjunto de las soluciones factibles para cada una de las dos variaciones del problema. Con este fin proponemos sendos modelos de programación lineal entera mixta, uno para cada variación. Ambos modelos comparten estructuras comunes: el conjunto de restricciones que define un camino hamiltoniano para cada día de servicio, el conjunto de restricciones de precedencia, que construyen de forma básica el vector de planificación, y finalmente, las restricciones que limitan la amplitud de la ventana temporal. Describiremos estas tres primeras familias de restricciones en primer lugar para, posteriormente, proponer una familia de restricciones específicas para cada variación del CTSP.

Definimos a continuación tres familias de variables compartidas por ambos modelos.

- Para cada día de servicio $k \in K$ y cada arco $(i, j) \in A(k)$, la variable binaria x_{ij} toma el valor 1 si durante el día de servicio k la ruta del vehículo alcanza la operación j inmediatamente después de haber realizado la operación i.

- Para cada operación i en O^+, la variable s_i representa el instante de tiempo en el que se inicia la operación i.

3.1. Restricciones comunes

Se describe a continuación las tres familias de restricciones que comparten ambas variaciones del problema.

3.1.1. Caminos hamiltonianos

Utilizaremos la notación estándard $A(S)$ en la descripción del siguientes restricciones, donde $S \subseteq V(k)$, para $k \in K$, para denotar el conjunto de arcos $(i, j) \in A(k)$ tales que $i, j \in S$.
Las siguientes restricciones describen $|K|$ caminos hamiltonianos, uno por cada día de servicio, en terminos de las variables x.

$$\sum_{(j,i)\in A(k)} x_{ji} = 1 \qquad j \in V(k), k \in K \tag{2}$$

$$\sum_{(i,j)\in A(k)} x_{ij} = 1 \qquad j \in V(k), k \in K \tag{3}$$

$$\sum_{(i,j)\in A(S)} x_{ij} \leq |S| - 1 \qquad S \subset V(k), k \in K \tag{4}$$

$$x_{ij} \in \{0, 1\} \qquad (i, j) \in A(k), k \in K \tag{5}$$

Las restricciones (2) y (3) garantizan que el vehículo visite exactamente una vez cada una de las localizaciones asociadas a las operaciones en $V(k)$ durante el día de servicio k. Las restricciones (4) evitan la existencia de subciclos en cada camino hamiltoniano. Finalmente, las restricciones (5) establecen que las variables x son binarias. Dada la definición del los conjuntos de arcos $A(k)$, para $k \in K$, las restricciones (2) y (3) garantizan que cada camino hamiltoniano comienze con cada operación de partida $(0, k)$ y concluya con cada la operación de llegada $(n + 1, k)$.

3.1.2. Restricciones de precedencia

Esta familia de restricciones determina el valor de las variables s en función de los caminos hamiltonianos descritos por las variables x. Podemos describir esta construcción en términos lógicos de la siguiente forma:

$$(x_{ij} = 1) \Rightarrow s_i + t_{ij} \leq s_j \quad (i,j) \in A(k), k \in K.$$

Esta expresión lógica establece que, para cada arco $(i,j) \in A(k)$ y cada día de servicio $k \in K$, si el arco (i,j) forma parte de la solución entonces la operación j debe ser llevada a cabo inmediatamente después de la operación i. Por tanto, el tiempo de servicio s_j de la operación j debe ser al menos la suma del tiempo de servicio s_i de la operación i más el tiempo de transporte entre ambas operaciones. Al establecer esta restricción que el valor de la derecha sea menor o igual al valor de la izquierda se permite la existencia de tiempos de espera entre las operaciones de entrega asociadas al mismo día de servicio.
Dado que esta expresión es inherentemente no lineal, la reformulamos de la siguiente forma utilizando el método de la M grande:

$$s_i + t_{ij} - M(1 - x_{ij}) \leq s_j \qquad (i,j) \in A(k), k \in K. \tag{6}$$

Donde M es un número suficientemente grande que garantiza que la restricción (6) se cumple si y solo si $x_{ij} = 1$. Estas restricciones se conocen en la literatura como desigualdades de Miller-Tucker-Zemlin (véase, por ejemplo, [3]). Como efecto colateral, la restricción (6) impide la formación de subciclos, convirtiendo en redundante en el caso entero las restricciones (4). Estas restricciones son válidas para ambas variaciones del CTSP, aunque para el caso específico del CTSP1 se requiere una restricción adicional que describiremos más adelante.

3.1.3. Limitación de los tiempos de servicio

La siguiente familia de restricciones establece que la diferencia entre los tiempos de servicio de dos operaciones de entrega asociadas al mismo cliente $l \in L$ no puede exceder la amplitud máxima w:

$$s_j - s_i \leq w \qquad (i,j) \in B(l), l \in L. \tag{7}$$

3.2. Restricciones específicas

A continuación describimos las restricciones específicas de cada una de las variaciones del CTSP.

3.2.1. CTSP sin tiempos de espera

Dado que el CTSP1 no admite tiempos de espera, la expresión lógica que describe las restricciones de precedencia se reformula de la siguiente forma:

$$(x_{ij} = 1) \Rightarrow s_i + t_{ij} = s_j \quad (i,j) \in A(k), k \in K.$$

En este caso, se establece una igualdad entre el lado derecho y el lado izquierdo de la restricción. Esto implica que el tiempo de servicio s_j de la operación j debe ser exactamente la suma del tiempo de servicio s_i de la operación i más el tiempo de transporte entre ambas operaciones.

Para conseguir representar esta expresión lógica en términos de desigualdades lineales, debemos completar la formulación del CTSP1 añadiendo a las restricciones (6) las siguientes restricciones:

$$s_i \geq s_j - t_{ij} - M(1 - x_{ij}) \qquad (i,j) \in A(k), k \in K. \tag{8}$$

La restricción (8) describe la desigualdad en el sentido contrario a (6), con el fin de implementar, entre ambas restricciones, la igualdad que se establece en la restricción lógica.

3.2.2. CTSP con tiempos de espera

La literatura (véase, por ejemplo, [2]) dota al CTSP2 con una restricción que limita el tiempo máximo de servicio desde la partida hasta la llegada al depósito. Podríamos describir estas restricciones en términos de las variables s de la siguiente forma:

$$s_{(n+1,k)} - s_{(0,k)} \leq T \qquad k \in K. \tag{9}$$

Donde T es un parámetro de entrada que representa el tiempo máximo de servicio desde la partida hasta la llegada al depósito.

3.3. Modelos matemáticos

Definamos el conjunto de arcos A como $A = \cup_{k \in K} A(k)$. Entonces, una vez descritas todas las familias de restricciones presentamos a continuación los modelos correspondientes a las dos variaciones del CTSP:

$$\text{CTSP1} = \left\{ \text{mín} \sum_{(i,j) \in A} c_{ij} x_{ij} \mid x \in \{(2)\text{–}(8)\} \right\}$$

$$\text{CTSP2} = \left\{ \text{mín} \sum_{(i,j) \in A} c_{ij} x_{ij} \mid x \in \{(2)\text{–}(7), (9)\} \right\}$$

Finalmente, merece la pena recordar que, **debido a que nuestro problema no admite tiempos de espera, la variante a resolver es el CTSP1.** Si es difícil encontrar una solución factible para CTSP, aún permitiendo tiempos de espera, lo será más para CTSP1.

4. Instancia a Resolver

Para probar nuestro algoritmo, se utilizará una batería de instancias del CTSP. A continuación, se explicarán las entradas y la forma de proceder con las mismas. Para ello, adjuntamos en la figura 3 un ejemplo real de una instancia del problema. Concretamente, burma14_p3_f50_lH, la misma que se ha utilizado para mostrar un ejemplo de solución factible.

```
NAME: burma14_p3_f50_lH
TYPE: CONTSP
COMMENT: 7328 (optimal value not allowing waiting), 7308 (optimal value
    allowing waiting)
DIMENSION: 14
NUM_DAYS: 3
DISTANCE: 2817
MAXIMUM_ALLOWABLE_DIFFERENTIAL: 617
EDGE_WEIGHT_TYPE: GEO
EDGE_WEIGHT_FORMAT: FUNCTION
DISPLAY_DATA_TYPE: COORD_DISPLAY
NODE_COORD_SECTION
   1  16.47       96.10
   2  16.47       94.44
   3  20.09       92.54
   4  22.39       93.37
   5  25.23       97.24
   6  22.00       96.05
   7  20.47       97.02
   8  17.20       96.29
   9  16.30       97.38
  10  14.05       98.12
  11  16.53       97.38
  12  21.52       95.59
  13  19.41       97.13
  14  20.09       94.55
DEMAND_SECTION
1 1 1 1
2 -1 -1 -1
3 1 1 -1
4 1 -1 1
5 1 1 1
6 1 -1 1
7 1 -1 1
8 -1 1 -1
9 1 1 -1
10 -1 -1 -1
11 -1 -1 -1
12 1 1 -1
13 -1 -1 1
14 -1 1 1
DEPOT_SECTION
1
-1
EOF
```

Figura 3: Ejemplo de instancia CTSP

Asimismo, se explica brevemente el significado de cada línea.

Campo	Descripción
NAME	Nombre de la instancia.
TYPE	Tipo de la instancia. Siempre será CTSP.
COMMENT	Contiene los valores óptimos para CTSP1 y CTSP2.
DIMENSION	Número total de clientes.
NUM_DAYS	Número de días de servicio.
DISTANCE	Tiempo total máximo. Despreciable para CTSP1.
MAXIMUM_ALLOWABLE_DIFFERENTIAL	Ventana temporal máxima.
EDGE_WEIGHT_TYPE	Métrica de distancia.
NODE_COORD_SECTION	Coordenadas de los clientes.
DEMAND_SECTION	Demanda de cada cliente en distintos días.
DEPOT_SECTION	Indica que cliente es el depósito. Típicamente, el 1.
-1	Final de la lista de depósitos.

Cuadro 2: Descripción general de los campos de la instancia CTSP

Algunos de estos campos requieren una explicación más detallada. En las siguientes subsecciones se describe su contenido con mayor profundidad.

4.1. Edge Weight Type

Para definir la distancia entre los clientes, se han utilizado distintas métricas de distancia. Es decir, para identificar la distancia real entre los clientes, no se ha hecho uso de una métrica fija, sino que se han impletenado varaias. Se han codificado la distancia de Manhattan y Euclidenia, entre otras muchas.

En el caso de la instancia $\mathrm{burma14_p3_f50_lH}$, se ha empleado la métrica Geográfica, y los campos `Edge_Weight_Format` y `Display_Data_Type`, son apartados específicos para la implementación de dicha métrica.

4.2. Demand Section

La sección `DEMAND_SECTION` especifica qué días requiere servicio cada cliente. Encontramos un valor para cada día de servicio y, en caso de ser un 1, el cliente requiere servicio en ese día, en caso de ser un -1 no lo requiere.
Por ejemplo:

- El **cliente 2** tiene valores $[-1, -1, -1]$, lo que significa que no requiere servicio en ningún día.
- El **cliente 3** tiene valores $[1, 1, -1]$, lo que indica que requiere servicio en el primer y segundo día, pero no en el tercero.

5. Solución

Una vez comprendido el problema, se presenta la solución propuesta. En particular, se ha desarrollado un algoritmo aproximado, cuya eficacia ha sido validada mediante 756 instancias adaptadas de la reconocida librería TSPLIB.

5.1. Descripción General del Algoritmo

A continuación, se añade una descripción muy general y resumida del algoritmo, el cual se irá definiendo más profundamente en los siguientes apartados.

En primer lugar, elaboramos una plantilla de precedencia, basada en la solución de un TSP, resuelto de forma heurística y considerando todos los clientes. Cabe mencionar que un TSP es un escenario del CTSP descrito anteriormente, donde tenemos un único día de servicio.

Para resolver el TSP, utilizaremos dos algoritmos constructivos diferentes, además de técnicas de perturbación e intensificación. Una vez hemos obtenido una solución final para el TSP, la utilizaremos para intentar resolver el CTSP1. Generaremos dos soluciones (una por heurística), donde cada una servirá como plantilla de precedencia inicial en la siguiente fase del algoritmo.

Partiendo de una solución del TSP, construimos una posible solución para el CTSP1, y evaluamos la factibilidad de la misma. En caso de haber encontrado una solución factible, intensificamos. Posteriormente, perturbamos e intensificamos nuevamente la solución del TSP, para generar nuevas posibles soluciones del CTSP1 y explorar en profundidad el espacio de soluciones. Procedemos de esta manera de forma iterativa.

Esta es una descripción muy general y resumida del algoritmo, a continuación se detallan las características más avanzadas, como, por ejemplo, la forma en la que se construye una solución del CTSP1 o cómo evaluar la factibilidad de la misma.

5.2. Solución TSP

Como se ha mencionado previamente, resolvemos un TSP considerando todos los clientes, es decir, teniendo en cuenta todos los objetivos en el campo de visión. Utilizaremos dos algoritmos constructivos diferentes, Cheapest Insertion y Nearest Neighbor. Estas heurísticas construyen una ruta paso a paso, añadiendo clientes de manera iterativa sin revisiones posteriores.

- Cheapest Insertion inserta en cada paso el cliente que cause el menor aumento en la distancia total, evaluando todas las posiciones posibles de inserción. Prioriza una solución más eficiente aunque con mayor coste computacional.
- Nearest Neighbor selecciona en cada paso el cliente más cercano al último añadido, lo que lo hace más rápido pero propenso a generar rutas de menor calidad.

Una vez obtenida la ruta inicial mediante un algoritmo constructivo, aplicamos una fase de intensificación seguida de un procedimiento de perturbación. En la fase de intensificación, utilizamos el método 2-opt *(First Improvement)* para explorar el entorno de la solución actual y obtener el óptimo local. Posteriormente, procedemos a perturbar la solución realizando un intercambio entre dos clientes seleccionados aleatoriamente, lo que constituye una perturbación leve. Esta

perturbación nos permite escapar de óptimos locales y explorar en mayor medida el espacio de soluciones.

Efectuamos 100 iteraciones de este proceso, en cada una de las cuales se intensifica la solución inicial y, posteriormente, se perturba para generar nuevas soluciones de partida. Una vez completadas las iteraciones, devolvemos la mejor solución encontrada.

5.2.1. 2-opt (First Improvement)

A continuación, se explica el algoritmo 2-opt, utilizado para la intensificación de una solución del TSP. Para una explicación más detallada del mismo, se puede consultar la siguiente referencia: https://link.springer.com/article/10.1007/s00453-013-9801-4.

La base del algoritmo consiste en identificar dos aristas no consecutivas, tales que al reconectar sus nodos se reduzca el coste total de la ruta, de tal manera que permite identificar cruces innecesarios y mejorar la calidad de la solución. Dadas dos aristas no consecutivas (v_i, v_{i+1}) y (v_j, v_{j+1}), sustituirlas por las aristas (v_i, v_j) y (v_{i+1}, v_{j+1}), invirtiendo el subcamino intermedio $(v_{i+1}, \dots, v_j)$, y comprobar si el cambio reduce el valor objetivo. Esto quiere decir, si

$$d(v_i, v_j) + d(v_{i+1}, v_{j+1}) \ < \ d(v_i, v_{i+1}) + d(v_j, v_{j+1}),$$

entonces aceptamos el cambio y actualizamos la ruta. Asimismo, es importante invertir el subcamino intermedio, ya que al hacerlo se restablece la secuencia adecuada de los vértices para garantizar una conexión directa entre v_i y $v_j + 1$. A continuación, se añade un ejemplo gráfico de una iteración de este algoritmo.

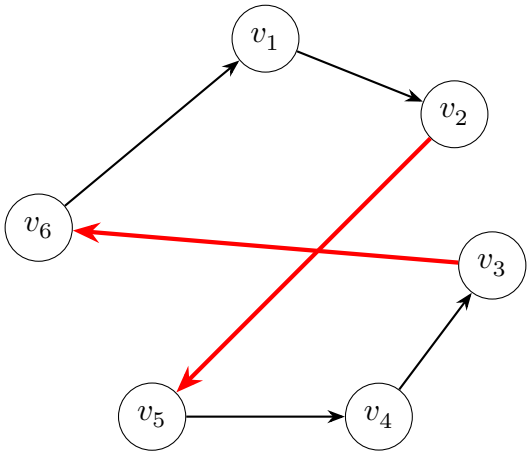


Figura 4: Ruta inicial TSP

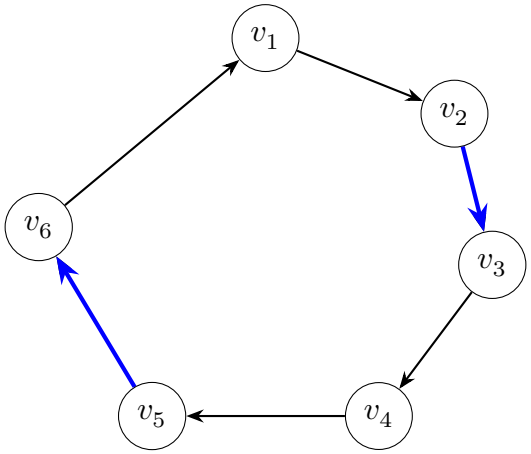


Figura 5: Ruta TSP tras reconexión con 2-opt

En este ejemplo, se han intercambiado las aristas (v_2, v_5) y (v_3, v_6), de tal manera que las nuevas aristas obtenidas son: (v_2, v_3) y (v_5, v_6). Una vez realizada la reconexión, estudiamos si el costo de las nuevas conexiones es menor, para identificar si hemos mejorado la solución inicial.

Asimismo, identificamos la importancia de invertir el camino intermedio. En nuestra solución inicial, encontramos: $(v_1) \to (v_2) \to (v_5) \to (v_4) \to (v_3) \to (v_6) \to (v_1)$. Al haber intercambiado las aristas (v_2, v_5) y (v_3, v_6), la nueva ruta es: $(v_1) \to (v_2) \to (v_3) \to (v_4) \to (v_5) \to (v_6) \to (v_1)$. Podemos apreciar que el camino intermedio, efectivamente, ha sido invertido, para mantener la coherencia de la ruta, evitar bucles y mantener una solución factible.

En relación a la implementación, el procedimiento de intensificación comienza inspeccionando todas las posibles reconexiones de aristas no consecutivas, partiendo desde la primera arista de la ruta. Si no se encuentra ninguna mejora, se continúa con la segunda arista, y así sucesivamente. Este proceso se repite hasta que se identifica una reconexión que reduce el coste total de la ruta. En tal caso, la nueva ruta obtenida pasa a ser la solución inicial, y el proceso de intensificación se reinicia desde el principio. Si se recorre toda la ruta sin encontrar mejoras, la solución actual se considera intensificada.

Se ha implementado la estrategia *First Improvement*, por lo que en nuestro proceso de intensificación, desde que encontramos una nueva solución que mejora la inicial, reiniciamos el proceso de intensificación partiendo de esa nueva solución. No se ha llevado a cabo la estrategia *Best Improvement*, donde en lugar de aplicar la primera mejora que se encuentra, se guarda la mejor ruta de todo el bucle para comenzar nuevamente el algoritmo a partir de dicha solución. A pesar de que este nuevo enfoque puede encontrar mejores soluciones (explora todo el vecindario antes de tomar decisiones), la técnica *First Improvement* es mucho más rápida e ideal para problemas grandes, donde revisar todas las reconexiones puede ser costoso. Asimismo, no es el objetivo de este trabajo encontrar la mejor solución del TSP posible, ya que una mejor solución del TSP no necesariamente generará una mejor solución para nuestro CTSP1, por lo que la estrategia *First Improvement* es la más adecuada para nuestro proyecto. Generará soluciones de gran calidad en un tiempo asequible.

5.3. Solución CTSP1

Para resolver el CTSP1, partimos de una solución para el TSP. Al haber resuelto el TSP, hemos obtenido una **plantilla de precedencia**. Ordenaremos los clientes a visitar en cada día de servicio, según el orden en el que aparezcan en la plantilla de precedencia. De esta manera, hemos planteado una ruta para cada uno de los días, donde en cada uno se visita a todos los clientes requeridos. Sin embargo, hay más restricciones a cumplir, como la ventana de tiempo máxima en la visita a un mismo cliente entre los distintos días de servicio. Esta posible solución del problema será corroborada por un verificador, cuyo funcionamiento se describe en el apartado 5.5.

Si el verificador aprueba la solución, significa que hemos encontrado una solución factible para nuestro problema. A continuación, aplicamos el proceso de intensificación (2-opt generalizado), descrito en el apartado 5.3.1, y la almacenamos. Si ya existía una solución previa, comparamos ambas y conservamos aquella con menor valor objetivo.

A continuación, independientemente de si la solución para el TSP nos permitió encontrar una solución factible o no, perturbaremos e intensificaremos dicha solución, para repetir el proceso pero partiendo de una plantilla de precedencia diferente. En este procedimiento, no buscamos necesariamente mejorar dicha solución, sino encontrar nuevas soluciones de calidad. De esta manera, utilizamos nuevas plantillas de precedencia para construir nuevas posibles soluciones para nuestro problema e inspeccionar en mayor medida el espacio de soluciones.

Iteraremos de este modo 650 veces, almacenando la mejor solución encontrada para el CTSP1. Esto quiere decir, aquella con menor costo total. Asimismo, el costo total de la ruta coincidirá con el tiempo total, dado que no se permiten tiempos de espera y los tiempos de visita son despreciados en el modelo, pues al ser constantes no modificarán el resultado de la optimización.

5.3.1. 2-opt Generalizado

A continuación, se explica el algoritmo 2-opt generalizado, empleado para mejorar la solución obtenida para el CTSP1.

Aplicaremos el algoritmo de intensificación 2-opt sobre la ruta descrita por el primer día de servicio. En caso de no encontrar una mejora, avanzamos al siguiente y así sucesivamente. Sin embargo, en caso de que encontremos una mejora en una ruta particular, realizaremos el mismo intercambio de esas dos aristas en todos los días de servicio que también contengan esas dos aristas de forma no consecutiva. Es decir, si para un día de servicio k, la reconexión de los nodos de las aristas (v_i, v_{i+1}), (v_j, v_{j+1}) reduce su costo total, para cualquier otro día k' que contenga dicho par de aristas de forma no consecutiva, también se le aplicará la misma reconexión. Una vez realizados todos los intercambios, evaluamos nuevamente la factibilidad de la solución, debido a que el cambio puede suponer que nuestra solución ya no sea factible, pues al cambiar ciertas rutas pueden no respetarse las ventanas temporales. En caso de que la solución no sea factible, continuamos con el algoritmo. En caso de que la solución sea factible, evaluamos el nuevo coste total. Si se ha reducido el valor objetivo, comenzamos el proceso de intensificación de nuevo, partiendo de esta nueva solución mejorada.

El algoritmo iterará de esta manera hasta no encontrar ninguna solución factible y con un costo menor que la solución de partida.

En este 2-opt generalizado también se ha optado por la estrategia *First Improvement*, ya que evaluar exhaustivamente todas las posibles reconexiones resulta computacionalmente muy costoso. Además, tras múltiples experimentaciones, se ha observado que en la mayoría de los casos las soluciones obtenidas con *First Improvement* coinciden con las de *Best Improvement*, ya que al aplicar repetidas perturbaciones y reoptimizaciones, se tiende a explorar las mismas regiones del espacio de búsqueda, de modo que ambas estrategias suelen ofrecer los mismos resultados.

5.4. Pseudocódigo

Se adjunta el pseudocódigo de manera muy resumida, pero suficientemente detallado para ayudar a entender el algoritmo previamente descrito.

5.4.1. Devolver Solución

Algoritmo llamado desde el programa principal, para retornar una solución factible. Cabe mencionar que el método IsFeas() se encarga de evaluar la solución del CTSP1 recibida por parámetros y devolver si es factible o no. IsFeas() es el verificador, aquí explicado: 5.5.

Algorithm 1 Build Solution

```
Entrada: No recibe parámetros
Salida: Solución CTSP                                   ▷ Devolvemos la mejor solución
function BuildSolution()
    TSP_sol1 ← TSP::CheapestInsertion(clients, adjacency_matrix)
    TSP_sol2 ← TSP::NearestNeighbor(clients, adjacency_matrix)
    CTSP_sol1 ← SolveFromTSP(TSP_sol1)
    CTSP_sol2 ← SolveFromTSP(TSP_sol2)
    if (IsFeas(CTSP_sol1) and IsFeas(CTSP_sol2)) then
        return (CTSP_sol1.cost() ≤ CTSP_sol2.cost() ? CTSP_sol1 : CTSP_sol2)
    end if
    if (IsFeas(CTSP_sol1) or IsFeas(CTSP_sol2)) then
        return (IsFeas(CTSP_sol1) ? CTSP_sol1 : CTSP_sol2)
    end if
    throw "No se ha encontrado solución"
end function
```

5.4.2. Construir Solución

Algoritmo utilizado para buscar la solución al CTSP partiendo de una solución para el TSP.

Algorithm 2 Solve From TSP

```
Entrada: TSP_sol: Solución Inicial TSP
Salida: Solución CTSP                                   ▷ Devolvemos la mejor solución
function SolveFromTSP(TSP_sol)
    best_sol ← BuildEmptySolMaxCost()
    for (i = 0 to max_iter) do
        CTSP_sol ← BuildSolFromTSPTemplate(TSP_sol)
        if (isFeas(CTSP_sol)) then
            CTSP_sol ← IntensificateSol(CTSP_sol)
            best_sol ← (best_sol.cost() ≤ CTSP_sol.cost() ? best_sol : CTSP_sol)
        end if
        TSP_sol ← TSP :: PerturbateTwoOpt(TSP_sol)
    end for
    return best_sol
end function
```

5.5. Verificador

A continuación, se explica detalladamente el funcionamiento del verificador, el cual nos permite identificar, a partir de la ruta asignada a cada uno de los días de servicio, si se trata de una solución factible o no. Para ello, debemos corroborar que se cumplen las dos restricciones fundamentales, por lo que cada cliente debe ser visitado en todos los días que requiere servicio y, además, se debe respetar la ventana temporal máxima entre la primera y última visita.

En primer lugar, comprobamos que todos los clientes han recibido servicio en todos los días que lo requieren. Haciendo uso de las rutas asignadas y conociendo la instancia del problema, esta implementación es trivial.

En segundo lugar, revisamos si se respetan las ventanas temporales. Para ello, calculamos el *starting time* de cada cliente, es decir, el instante en el que cada localización es visitada por cada día de servicio. Así, para cada día k, determinamos en qué momento específico visita cada cliente l.
El cálculo de $s_{(l,k)}$ representa el instante en el que el cliente l es visitado en el día k. Asimismo, entendemos $i = (l, k)$ como la operación asociada al cliente l con el día k. Dado que en todos los días de servicio se parte del depósito (denominado cliente 0, típicamente), el tiempo de inicio de la visita al primer cliente dependerá únicamente del tiempo de traslado desde el depósito hasta dicho objetivo.
Por conveniencia, consideramos que $s_{(0,k)} = 0$, ya que el instante de partida es siempre el mismo. Esto implica que el tiempo de salida del depósito es el mismo para cualquier día k, por lo que no afecta al análisis de las ventanas temporales. Como resultado, el *starting time* del primer cliente a visitar se reduce simplemente al tiempo de traslado desde el depósito hasta él. Siendo el cliente 0 el depósito y l el primer objetivo a visitar, obtenemos:

$$s_{(l,k)} = t_{(0,k),(l,k)}$$

Para los clientes siguientes, el tiempo de visita se obtiene sumando el *starting time* del cliente anterior con el tiempo de traslado entre ambos. Consecuentemente, si existe una conexión directa entre $i = (l, k)$ y $j = (l', k)$, donde l' representa el segundo cliente a visitar, obtenemos:

$$s_j = s_i + t_{i,j}$$

De esta manera, el *starting time* de cada localización se obtiene de forma iterativa, conociendo únicamente el instante en el que se visitó al cliente previo y el tiempo de traslado hasta el siguiente. Esta metodología nos permite calcular de manera eficiente los tiempos de visita de todos los clientes en cada día de servicio.
Una vez comprendida la forma de operar, determinar si se cumplen las ventanas temporales para cada cliente es fácil de entender. Al conocer en qué instante es visitado cada cliente en cada día, identificamos para el mismo cliente el tiempo de visita menor y mayor. Si la diferencia entre ambos es menor o igual que la ventana temporal máxima, podemos afirmar que se cumple dicha restricción, pues la diferencia de tiempo entre la visita más temprana y más tardía respeta dicha ventana. En caso de ser mayor, no se respeta la ventana temporal y no es una solución factible.

6. Visualización Resultados Obtenidos

A continuación se realizará un estudio del algoritmo codificado, analizando los resultados obtenidos tras ser validado mediante 756 instancias. Asimismo, dichas instancias han sido agrupadas en 42 familias diferentes, de tal manera que todas comparten propiedades como origen, métrica de distancia, rasgos de dimensión o características intrínsecas del problema (asimetría, ausencia de métrica). Cabe mencionar que para cada familia de instancias, encontramos un total de 18 entradas.

Para evaluar la calidad de las soluciones obtenidas mediante el algoritmo heurístico, se utilizará una medida denominada ***gap***. Esta representa el porcentaje en que la solución heurística excede al valor de la solución óptima, y se calcula mediante la siguiente expresión:

$$\text{gap} = \frac{\text{valor heurístico} - \text{valor óptimo}}{\text{valor óptimo}} \times 100$$

De este modo, un valor del $10\,\%$ indica que la solución proporcionada por el algoritmo heurístico supera en un $10\,\%$ al valor óptimo. En cambio, un valor del $0\,\%$ implica que se ha alcanzado la solución óptima.

6.1. Análisis por Familia de Instancias

En primer lugar, estudiamos los resultados del algoritmo agrupando las instancias por familia. De esta manera, para cada familia de instancias, calcularemos los valores medios de tiempo de ejecución y gap. En la última columna encontramos el número de soluciones encontradas, **siendo 18 el número total de instancias por familia.**

Familia	T(s)	Gap	Sol. Encontradas
att48	5.06	3.56 %	17
bayg29	0.93	4.05 %	16
bays29	1.09	1.56 %	16
berlin52	6.59	0.78 %	17
br17	0.14	0.00 %	18
brazil58	9.65	1.40 %	16
burma14	0.11	0.31 %	18
dantzig42	3.41	1.57 %	18
eil101	53.53	5.59 %	18
eil51	5.94	5.49 %	17
eil76	21.64	4.63 %	18
fri26	0.69	1.44 %	18
ft53	6.36	17.56 %	17
ft70	15.02	13.80 %	16
ftv33	1.79	5.11 %	18
ftv35	1.87	7.31 %	17
ftv38	2.43	9.10 %	18
ftv44	3.55	9.79 %	17
ftv47	4.60	24.86 %	17
ftv55	7.15	12.99 %	15
ftv64	11.34	23.53 %	17
ftv70	15.01	21.42 %	16
gr17	0.18	1.65 %	17
gr21	0.35	1.12 %	17
gr24	0.56	3.41 %	16
gr48	5.20	3.65 %	18
gr96	49.30	3.32 %	16
hk48	5.10	1.93 %	18
kroA100	56.06	1.65 %	18
kroB100	56.94	3.47 %	18
kroC100	56.37	1.81 %	18
kroD100	56.80	2.51 %	18
kroE100	56.73	4.25 %	18
p43	4.36	0.36 %	18
pr76	22.92	1.61 %	17
rat99	52.82	2.07 %	18
rd100	55.33	2.33 %	18
ry48p	5.28	4.61 %	17
st70	17.14	2.43 %	17
swiss42	3.31	1.08 %	18
ulysses16	0.15	0.94 %	15
ulysses22	0.45	0.28 %	18

Cuadro 3: Resultados del algoritmo, agrupando por familias de instancias.

Estudiamos cuántas familias se encuentran en un rango específico de gap. Además, calculamos la moda de la columna Soluciones Encontradas. Se utiliza *G* como abreviatura de gap, para estilizar la tabla.

	G $\in [0\,\%, 1\,\%)$	G $\in [1\,\%, 5\,\%)$	G $\in [5\,\%, 10\,\%)$	G $\geq 10\,\%$	**Mo**
Familia Instancias	6	24	6	6	
Soluciones Encontradas					18

Cuadro 4: Rango de Gaps y Moda Soluciones Encontradas.

Por un lado, apreciamos que la moda para la variable Soluciones Encontradas es 18, lo cual significa que el **comportamiento más frecuente es resolver el 100 % de las instancias de una familia.**

Por otro lado, observamos que para la mayoría de las familias las soluciones encontradas son muy satisfactorias. Para un total de 6 familias se ha encontrado prácticamente la solución óptima. El comportamiento más habitual es obtener un Gap $\in [5\,\%, 10\,\%)$, por lo que no excede en demasía el valor de la solución óptima. No obstante, para 6 de ellas encontramos Gap $\geq 10\,\%$, evidenciando que hay un grupo de familias donde los resultados difieren notablemente.

Observando la tabla, las familias de instancias que suponen un reto mayor para nuestro algoritmo son `ft53, ft70, ftv47, ftv55, ftv64 y ftv70`.

6.1.1. Complejidad Familias Ft

No es casualidad que las familias de instancias con un gap mayor sean las instancias `ft`. Estas presentan una complejidad mayor, **debido a la asimetría y ausencia de métrica**, lo cual indica que no se trata de un CTSP métrico, perjudicando notoriamente a nuestro algoritmo.

Un problema de CTSP se dice métrico si la función de coste $d : V \times V \to \mathbb{R}_{\geq 0}$ satisface para todo $i, j, k \in V$ las siguientes condiciones:

1. **No negatividad e identidad:**

$$d(i,j) \geq 0, \quad d(i,i) = 0.$$

2. **Simetría:**

$$d(i,j) = d(j,i).$$

3. **Desigualdad triangular:**

$$d(i,k) \leq d(i,j) + d(j,k).$$

La tercera condición, la desigualdad triangular, garantiza que el camino directo entre dos nodos siempre tendrá un coste menor que tomar un camino intermedio. Sin embargo, **las condiciones de simetría y desigualdad triangular no se cumplen para la familia `ft`.**

En las familias `ft`, los costes se dan como una `FULL_MATRIX` de valores enteros explícitos. Esto quiere decir que cada valor $d(i,j)$ aparece directamente en la matriz, sin hacer referencia a

ningún sistema de coordenadas. **Las distancias aparecen como valores listados arbitrariamente en una matriz completa, sin derivar de coordenadas reales ni de ninguna función geométrica estándar**. Consecuentemente, pueden violarse tanto la condición de simetría como desigualdad triangular. Por ejemplo, podría darse el siguiente escenario. Considerando tres nodos i, j, k con costes:

$$d(i,j) = 30, \quad d(j,i) = 20, \quad d(i,k) = 50, \quad d(j,k) = 15 \quad d(i,j) + d(j,k) = 30 + 15 = 45$$

Observamos que:

- La simetría se viola porque $d(i,j) = 30 \neq 20 = d(j,i)$.
- La desigualdad triangular no se cumple porque

$$d(i,k) = 50 > 45 = d(i,j) + d(j,k).$$

Este comportamiento afecta directamente a nuestro algoritmo, ya que tiene un impacto negativo en las heurísticas de construcción e intensificación.

Por un lado, la violación de la desigualdad triangular rompe las suposiciones fundamentales que utilizan los métodos de Nearest Neighbor o Cheapest Insertion para seleccionar arcos de forma eficiente.

- Nearest Neighbor incorpora el cliente no visitado más cercano al último cliente añadido, asumiendo así que este nuevo arco no penalizará en exceso las reconexiones posteriores. Sin embargo, sin desigualdad triangular, un arco localmente corto puede resultar en conexiones posteriores que aumenten signficamente el coste total de la ruta. Es decir, añadir el nodo más cercano al último añadido, puede llevar a un salto que, al reconectar con el resto de la ruta, sea más caro que desviarse por otros vértices intermedios.
- Cheapest Insertion añade en cada paso el cliente que cause el menor aumento en la distancia total. No obstante, al no cumplirse la desigualdad triangular, la inserción que parece más barata en un paso puede forzar luego arcos muy costosos al reconectar la ruta completa, ya que la heurística descarta posiciones que, aunque inicialmente más caras, habrían permitido atajos globales más económicos.

Por otro lado, las técnicas de intensificación 2-opt y 2-opt generalizado pierden la propiedad de garantizar mejoras locales, al no cumplirse las propiedades métricas. Al eliminar dos aristas y reconectar los cuatro extremos, se asume que la suma de los nuevos arcos es menor que la de los eliminados si éstos eran cruzados. Sin embargo, en ausencia de desigualdad triangular, este intercambio puede incluso aumentar la longitud de la ruta. Asimismo, debido a la falta de simetría, invertir el subcamino intermedio modifica absolutamente el valor objetivo de la solución.

A modo de conclusión, como consecuencia de la asimetría y ausencia de métrica, las heurísticas de construcción e intensificación pierden propiedades y garantías de cercanía al óptimo.

6.2. Análisis por Número de Clientes

A continuación, agrupamos por número de clientes.

Clientes	T(s)	Gap	Sol. Encontradas	Instancias	Prob. solución factible
13	0.11	0.31 %	18	18	100.00 %
15	0.15	0.94 %	15	18	83.33 %
16	0.16	0.80 %	35	36	97.22 %
20	0.35	1.12 %	17	18	94.44 %
21	0.45	0.28 %	18	18	100.00 %
23	0.56	3.41 %	16	18	88.89 %
25	0.69	1.44 %	18	18	100.00 %
28	1.01	2.81 %	32	36	88.89 %
33	1.79	5.11 %	18	18	100.00 %
35	1.87	7.31 %	17	18	94.44 %
38	2.43	9.10 %	18	18	100.00 %
41	3.36	1.32 %	36	36	100.00 %
42	4.36	0.36 %	18	18	100.00 %
44	3.55	9.79 %	17	18	94.44 %
47	5.05	7.61 %	87	90	96.67 %
50	5.94	5.49 %	17	18	94.44 %
51	6.59	0.78 %	17	18	94.44 %
52	6.36	17.56 %	17	18	94.44 %
55	7.15	12.99 %	15	18	83.33 %
57	9.65	1.40 %	16	18	88.89 %
64	11.34	23.53 %	17	18	94.44 %
69	16.08	7.94 %	33	36	91.67 %
70	15.01	21.42 %	16	18	88.89 %
75	22.28	3.16 %	35	36	97.22 %
95	49.30	3.32 %	16	18	88.89 %
98	52.82	2.07 %	18	18	100.00 %
99	56.37	2.67 %	108	108	100.00 %
100	53.53	5.59 %	18	18	100.00 %

Cuadro 5: Resultados del algoritmo, agrupando por número de clientes.

Estudiamos la relación entre el gap y el número de clientes, para identificar si la calidad de las soluciones de nuestro algoritmo está condicionada por el número de clientes.

Figura 6: Reresentación gráfica gap frente al número de clientes

Apreciamos que no existe una relación directa entre el número de clientes y el gap, pues el incremento significativo de uno no implica el aumento del otro. Encontramos que para 52, 64 y 70 clientes los resultados difieren notablemente del resto. Sin embargo, este aumento en el gap no se debe realmente al número de objetivos a visitar, se debe a que las familias `ft53`, `ftv64` y `ftv70` son aquellas que incorporan precisamente ese número de localizaciones. **Consecuentemente, el valor del gap no está condicionado por el número de clientes, pero sí por la familia de instancias.**

Por otro lado, **aumentar el número de clientes sí incrementa el tiempo de ejecución,** puesto que las rutas incorporan un mayor número de nodos y aumenta notablemente el coste computacional para crear soluciones e intensificarlas.

Figura 7: Reresentación gráfica tiempo ejecución frente al número de clientes

Finalmente, **aumentar el número de clientes tampoco influye en la probabilidad de encontrar una solución factible,** pues no hay una relación directa entre estas dos variables. Para aquellos escenarios donde la probabilidad de encontrar una solución factible es menor que el resto, no es debido al número de clientes, sino debido a las familias `ft`.

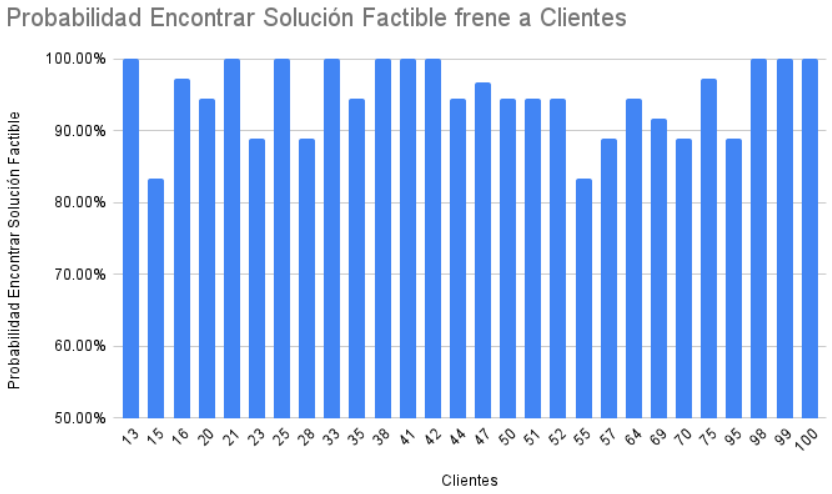


Figura 8: Reresentación gráfica probabilidad encontrar solución factible frente al número de clientes

6.3. Análisis por Número de Tareas

Agrupamos los resultados del algoritmo por el número de tareas, es decir, por el número de servicios a programar. Previamente se ha corroborado que aumentar el número de clientes no empeora los resultados, pero incrementar el número de clientes no implica necesariamente acrecentar el número de tareas, pues podría darse el extraño caso de tener muchos clientes que no reciban servicio.

Tareas	T(s)	Gap	Sol. Encontradas	Instancias	Prob. solución factible
[0-50]	0.30	1.19 %	74	78	94.87 %
[51-100]	2.27	4.64 %	175	183	95.63 %
[101-150]	6.23	7.47 %	149	156	95.51 %
[151-200]	19.55	7.20 %	122	132	92.42 %
[201-250]	30.38	5.48 %	74	78	94.87 %
[251-300]	44.92	3.91 %	60	60	100.00 %
[301-350]	30.30	6.01 %	18	18	100.00 %
[351-400]	50.54	2.39 %	24	24	100.00 %
[401-450]	52.79	0.46 %	6	6	100.00 %
[451-500]	55.53	0.99 %	21	21	100.00 %

Cuadro 6: Resultados del algoritmo, agrupando por número de tareas.

Comprobamos que un mayor número de tareas no implica un empeoramiento de los resultados. Tal como era de esperar, el tiempo medio de ejecución aumenta, pero no se observa una disminución ni en la cantidad de soluciones encontradas ni en su calidad. El gap se mantiene en valores muy satisfactorios, al igual que la probabilidad de encontrar una solución factible.

No obstante, encontramos un comportamiento aparentemente extraño. Para el rango de [101-200] tareas encontramos un gap superior al resto. Podría influir que en este rango trabajamos con más instancias, pero para el rango de [51-100] tareas también validamos el comportamiento para un gran número de entradas. El número de tareas no parece ser el factor que esté desmejorando los resultados, pues no tendría sentido obtener un comportamiento peor justo para un rango intermedio de tareas. Comprobamos si las familias `ft` más complicadas son la razón. Utilizamos *Ta* como abreviatura de tareas, para estilizar la tabla.

	Ta $\in [0, 100]$	Ta $\in [101, 200]$	Ta $\in [201, 300]$	Ta $\in [301, 350]$	Ta > 350
ftv47	3	12	3	0	0
ft53	3	12	3	0	0
ftv55	3	9	6	0	0
ftv64	0	12	6	0	0
ft70	0	12	3	3	0
ftv70	0	12	3	3	0
Total	9	69	24	6	0

Cuadro 7: Rango de tareas para las familias ft.

Se observa que, precisamente en el rango de [101–200] tareas, se concentra el mayor número de instancias pertenecientes a estas familias `ft`. Concretamente, 69 instancias. El número total de entradas en dicho rango es de 288, por lo que las instancias de la familia `ft` forman un 23.96 % del total. Consecuentemente, **podemos explicar la ligera desmejora de nuestro algoritmo para este número de tareas, debido a la notable presencia de instancias `ft` en este rango**.

Por otro lado, para el rango de [300-350] tareas, encontrábamos un gap del 6.01 %, siendo este también superior a la media. Sin embargo, estas familias `ft` cuentan únicamente con 6 instancias dentro de este rango. No obstante, el número total de instancias entre 300 y 350 tareas es menor que en otros muchos intervalos, pues cuenta solamente con 18 entradas. **Consecuentemente, las instancias `ft` pertenecientes a estas familias protagonizan el $33.\overline{3}\,\%$ de entradas en este rango, explicando nuevamente el aumento en el valor del gap.**

6.3.1. Impacto Ventana Temporal Máxima

Una vez analizado el comportamiento atendiendo al número de tareas, es imprescindible profundizar en el impacto de la ventana temporal máxima. Con ventanas muy generosas, el margen de maniobra es más amplio, por lo que añadir servicios resulta más sencillo. Sin embargo, si estrechamos ese límite, la dificultad para encontrar soluciones que cumplan con las restricciones del problema puede aumentar considerablemente. Para estudiar si nuestro algoritmo funciona satisfactoriamente incluso con ventanas temporales muy bajas, comprobamos los resultados para instancias que tengan el mismo número de tareas, clientes y días de servicio. Concretamente, 175 tareas, 99 clientes y 3 días de servicio. Únicamente variará la ventana temporal máxima.

Ventana Temporal Máxima	T(s)	Gap	Solución Encontrada
618	56.004	2.00 %	1
927	55.231	2.01 %	1
1237	56.505	2.18 %	1
1705	56.48	1.92 %	1
1727	55.431	3.25 %	1
1846	57.502	6.36 %	1
2558	55.84	1.92 %	1
2590	55.979	3.42 %	1
2769	56.682	6.72 %	1
3411	55.547	1.92 %	1
3454	55.474	3.47 %	1
3693	56.459	6.72 %	1

Cuadro 8: Impacto ventanta temporal máxima.

Apreciamos que se ha encontrado una solución factible para todas las ventanas temporales. Por otro lado, no parece haber ninguna relación directa entre el gap y la ventana temporal máxima. Visualizamos los datos gráficamente para interpretarlos más fácilmente.

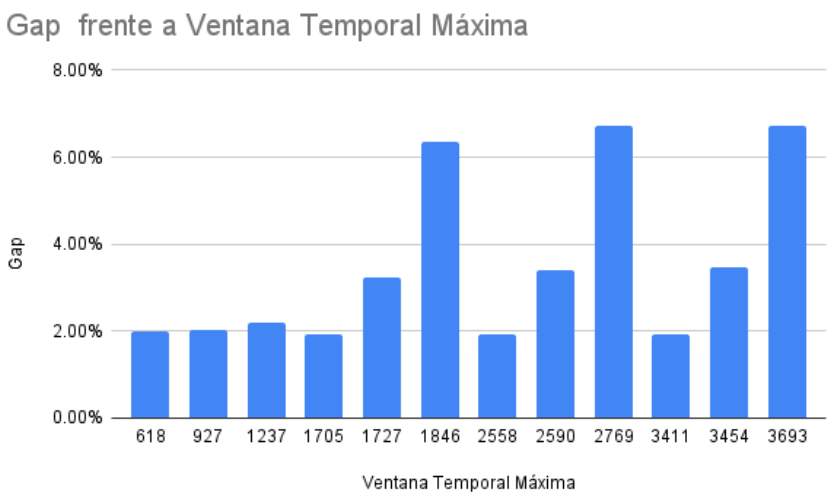


Figura 9: Reresentación gráfica gap frente a ventanta temporal máxima

Comprobamos que disminuir la ventana temporal máxima no afecta negativamente al comportamiento. Los resultados obtenidos no guardan ninguna relación directa con la ventana temporal máxima, pues a pesar de tener el mismo número de clientes, días de servicio y tareas, al disminuirla no empeoran los resultados. **Consecuentemente, el algoritmo funciona satisfactoriamente, adaptándose a diferentes ventanas temporales y encontrando soluciones de calidad independientemente de la misma.**

6.4. Análisis por Número de Días de Servicio

A continuación, analizamos la relevancia de aumentar o disminuir los días de servicio y su efecto en el algoritmo codificado.

Estudiamos las diferencias en los resultados obtenidos para instancias con el mismo número de clientes, pero modificando los días de servicio. El número de tareas se mantendrá relativamente proporcional, puesto que para las instancias con el mismo número de clientes, al aumentar el número de días de servicio también incrementará el número de tareas.

Clientes	Días Servicio	T(s)	Gap	Sol. Encontradas	Instancias	Prob. sol. factible
25	3	0.68	1.01 %	9	9	100.00 %
	5	0.70	1.86 %	9	9	100.00 %
50	3	5.90	4.31 %	9	9	100.00 %
	5	5.97	6.82 %	8	9	88.89 %
75	3	22.36	3.52 %	17	18	94.44 %
	5	22.19	2.83 %	18	18	100.00 %
100	3	53.50	4.94 %	9	9	100.00 %
	5	53.55	6.24 %	9	9	100.00 %

Cuadro 9: Resultados del algoritmo, agrupando por días de servicio y número de tareas.

Apreciamos que aumentar el número de días de servicios no tiene un impacto negativo en los resultados. Todos los parámetros presentan unos valores muy similares, por lo que, a pesar de incrementar el número de días de servicio, las soluciones continúan siendo de calidad. Consecuentemente, el algoritmo es capaz de manejar el número de días de servicio muy satisfactoriamente.

6.5. Análisis Global

A continuación, se adjunta una tabla con los resultados medios del algoritmo.

Instancias	T(s)	Gap	Sol. Factible	Sol. Óptima	Gap <= 1 %
756	16.26	5.15 %	95.63 %	38	31.5 %

Cuadro 10: Resultados medios.

Concluimos que los resultados son muy satifactorios. Obtenemos un tiempo de ejecución más que razonable, un gap adecuado y una probabilidad para encontrar una solución factible muy alta. Asimismo, se ha encontrado la solución óptima hasta en 38 ocasiones y un gap <= 1 % para el 31.5 % de las instancias. **Consecuentemente, el algoritmo funciona de manera muy positiva, encontrando soluciones de calidad en la mayoría de los casos**.

Por otro lado, realizamos un análisis de los resultados medios únicamente para CTSP métricos (excluyendo las familias `ft`).

Instancias	T(s)	Gap	Sol. Factible	Sol. Óptima	Gap <= 1 %
576	19.13	2.22 %	96.18 %	38	40.63 %

Cuadro 11: Resultados medios con CTSP métrico.

Apreciamos que **únicamente para CTSP métricos los resultados son aún mejores. Un gap medio del 2.22 % es un valor muy positivo, teniendo en cuenta un tiempo de ejecución medio menor a 20 segundos**. Esta mejoría era previsible, pues, como ha sido mencionado anteriormente, las heurísticas de construcción e intensificación pierden garantías para CTSP no métricos. Sin embargo, para el escenario más típico y frecuente (CTSP métrico), los resultados son muy satisfactorios.

Finalmente, estudiamos el comportamiento únicamente para CTSP no métricos.

Instancias	T(s)	Gap	Sol. Factible	Sol. Óptima	Gap <= 1 %
180	6.91	14.44 %	93.33 %	0	2.22 %

Cuadro 12: Resultados medios CTSP no métrico.

Como era previsible, se puede observar que el gap medio aumenta y la probabilidad de encontrar una solución factible disminuye. **No obstante, atendiendo a la gran dificultad que ofrecen este grupo de familias, encontrar soluciones factibles para el 93.33 % de las instancias con un gap medio de 14.44 % continúa siendo unos grandes resultados**, pues respaldan la capacidad del algoritmo para encontrar soluciones factibles, incluso para instancias asimétricas y con ausencia de métrica.

6.6. Justificación Número Iteraciones

Comparamos los resultados obtenidos modificando drásticamente el número de iteraciones, para de esta manera justificar los valores empleados. Analizamos los siguientes resultados para **las primeras 473 entradas**. No se ha utilizado la batería de instancias en su completitud, dado que al aumentar el número de iteraciones, el tiempo de ejecución aumenta exponencialmente.

En primer lugar, estudiamos los resultados de nuestro algoritmo. Se recuerda que el número de iteraciones para construir soluciones del TSP y CTSP ha sido de 100 y 650 iteraciones, respectivamente.

T(s)	MAX T(s)	Gap	Sol. Factible	Sol. Óptima	Gap <= 1 %
4.37	17.30	6.67 %	94.44 %	36	29.06 %

Cuadro 13: Resultados algoritmo, primeras 473 instancias

En segundo lugar, aumentamos exageradamente el número de iteraciones, para comprobar si tiene un impacto considerable en los resultados. Se adjuntan los resultados para 1000 y 5000 iteraciones en la construcción de soluciones para TSP y CTSP.

T(s)	MAX T(s)	Gap	Sol. Factible	Sol. Óptima	Gap <= 1 %
115.28	434.89	5.54 %	95.76 %	37	30.82 %

Cuadro 14: Resultados algoritmo modificando número de iteraciones, primeras 473 instancias

Estudiando ambas tablas, apreciamos una enorme diferencia en el tiempo de ejecución. Con el número de iteraciones utilizado, el tiempo medio no supera los 5 segundos y, para resolver las instancias más complejas de estas 473, no supera los 18 segundos. Sin embargo, al aumentar notoriamente el número de iteraciones, se obtiene un tiempo medio de prácticamente 2 minutos y, para las entradas con mayor número de clientes y tareas, ha superado los 7 minutos. Asimismo, dado que estos tiempos resultan ya considerablemente elevados incluso en instancias de coste moderado, cabe prever que las entradas más complejas exigirían un tiempo de ejecución inadmisible, lo que imposibilita llevar a cabo un estudio exhaustivo con la batería completa de casos.

Por otro lado, a pesar de esta gran diferencia en el tiempo de ejecución, los resultados obtenidos no mejoran notablemente. El gap medio se ha reducido únicamente en un 1 % y la probabilidad de encontrar soluciones factibles ha aumentado en un 1 %. **Consecuentemente, aumentar el número de iteraciones incrementa exageradamente el tiempo de ejecución. Sin embargo, la mejora en la calidad de la soluciones es realmente mínima.** Además, si no se ha hallado una diferencia notable en los resultados incrementando de manera tan drástica el número de iteraciones, tampoco se hará modificando estos valores en menor medida. Para identificar una mínima mejora en los resultados, el tiempo de ejecución debe aumentar de manera insostenible.

Asimismo, merece la pena mencionar que no se ha disminuido el número de iteraciones para aminorar el tiempo de ejecución, puesto que los resultados obtenidos presentan unos valores muy satisfactorios con un tiempo de ejecución más que positivo. Decrementar el número de iteraciones podría reducir el tiempo de ejecución, ofreciendo unos resultados ligeramente peores.

Sin embargo, aunque la diferencia en los resultados sea mínima, no se encuentra la necesidad de reducir el tiempo de ejecución y empeorar las soluciones.

En último lugar, gracias a este estudio, **podemos concluir que para mejorar los resultados de nuestro algoritmo no es necesario aumentar el número de iteraciones, sino añadir reconfiguraciones de rutas más profundas para explorar en mayor medida el espacio de soluciones**.

7. Conclusiones y Líneas Futuras

Finalmente, podemos concluir que los objetivos iniciales han sido cumplidos con éxito.

En primer lugar, el algoritmo ha sido validado mediante 756 instancias adaptadas de la reconocida librería TSPLIB, con unos resultados muy satisfactorios. Independientemente del número de clientes, días de servicio, tareas o ventana temporal máxima, ha encontrado soluciones de calidad, manteniendo un tiempo de ejecución reducido. El factor condicionante para el rendimiento del mismo ha sido la familia de instancias. Los resultados obtenidos para las familias `ft` difieren del resto, debido a la asimetría y ausencia de métrica. No obstante, se ha podido resolver la inmensa mayoría de instancias para estas familias, manteniendo un tiempo de ejecución y gap razonables. Debido a la gran complejidad que supone este tipo de instancias, la capacidad para encontrar tantas soluciones factibles en un tiempo de ejecución asumible supone un buen resultado. Por otro lado, el comportamiento obtenido para CTSP métricos (el tipo de instancia más habitual) es muy prometedor, con un gap del 2 % aproximadamente. **Consecuentemente, el análisis de los resultados es realmente positivo.**

En segundo lugar, **también se han cumplido el resto de objetivos iniciales.** Esto quiere decir, la descripción formal del problema, la elaboración de un *parser* para la lectura de una instancia CTSP a un objeto *instance* en C++, la visualización gráfica de soluciones obtenidas y un análisis de las mismas.

En último lugar, en lo que a líneas futuras se refiere, encontramos la posibilidad de utilizar otros operadores de intensificación, diferentes a la 2-optimalidad, para de esta manera explorar nuevas configuraciones del espacio de soluciones y potencialmente mejorar la calidad de los resultados obtenidos. Entre estos operadores, podríamos considerar variantes más avanzadas como 3-opt o k-opt, que permiten reconfiguraciones más profundas de la ruta. Asimismo, para abordar de forma más efectiva instancias del CTSP no métricas, sería interesante adaptar los algoritmos implementados, de tal manera que no asuman la validez de la desigualdad triangular, evaluando siempre el coste real tras cada modificación. Esto permitiría considerar rutas que, bajo suposiciones métricas, serían descartadas pero que podrían resultar beneficiosas en contextos no métricos. Además, podría incorporarse un mecanismo que, ante la inserción de una arista especialmente costosa, examine si habría sido más eficiente alcanzar el mismo destino mediante uno o varios nodos intermedios. El tiempo de ejecución aumentaría significativamente, sin embargo, actualmente tiene un valor muy reducido y podría asumirse aumentar el coste computacional.

8. Summary and Conclusions

As a conclusion, the initial objectives have been satisfactorily achieved, since the problem **MIRADAS** has been understood, formulated, and solved. To this end, the **Consistent Traveling Salesperson Problem (CTSP) has been used as an abstraction of the original problem**, specifically, a variant that does not allow waiting times. Consequently, if finding a solution for the CTSP is already challenging, it is even more so for this no-wait variant. The coded algorithm, based on precedence templates, added to the constructive and intensification heuristics, has enabled a suitable exploration of the solution space, yielding more than positive results.

On the other hand, **regarding the algorithm's performance,** it has been validated on 756 instances from the TSPLIB library, producing feasible solutions in 95.63 % of cases while maintaining a distance to the optimal solution that is more than acceptable. Moreover, it has proven capable of solving the most complex instances, thereby demonstrating its ability to handle non-metric CTSP, despite the additional difficulty posed by asymmetry and the lack of metric properties.

Furthermore, in addition to the aforementioned achievements, **all remaining objectives have been met,** including the development of a parser for reading a CTSP instance into a C++ instance object, the graphical visualization of the obtained solutions, and an analysis thereof.

9. Presupuesto

A continuación, se detalla el presupuesto completo de este Trabajo Fin de Grado. Cabe mencionar que no ha sido necesario el pago de licencias, puesto que todas las instancias de prueba han sido obtenidas de manera gratuita a través de la librería TSPLIB.

Descripción	**Cantidad**	**Coste (€)**
Portátil	1	1200,00
Licencias Necesarias	0	0,00
Subtotal de Equipos y Licencias		**1200,00**

Cuadro 15: Presupuesto de Equipos y Licencias

Descripción	**Horas**	**Coste (€)**
Precio por Hora		10,00
Total de Horas de Trabajo	300	
Costo Total del Trabajo Humano		**3000,00**

Cuadro 16: Coste de Mano de Obra

Descripción	**Coste Total (€)**
Subtotal de Equipos y Licencias	1200,00
Costo Total del Trabajo	3000,00
Coste Total del Proyecto	**4200,00**

Cuadro 17: Coste Total del Proyecto

10. Bibliografía y Referencias

1. Instituto de Astrofísica de Canarias. MIRADAS. Mid-resolution InfRAreD Astronomical Spectrograph.
   URL: https://www.iac.es/es/proyectos/miradas

2. A. Subramanyam, C. E. Gounaris, A branch-and-cut framework for the consistent traveling salesman problem, Eur. J. Oper. Res. 248 (2016) 384–395.
   URL: https://doi.org/10.1016/j.ejor.2015.07.030.

3. A. Subramanyam, C. E. Gounaris, A decomposition algorithm for the consistent traveling salesman problem with vehicle idling, Transportation Sci. 52 (2018) 386–401.
   URL: https://doi.org/10.1287/trsc.2017.0741.

4. L. Gouveia, J. M. Pires, The asymmetric travelling salesman problem and a reformulation of the miller–tucker–zemlin constraints, Eur. J. Oper. Res. 112 (1999) 134–146.
   URL: https://doi.org/10.1016/S0377-2217(97)00358-5.

5. Matthias Englert, Heiko Röglin, Berthold Vöcking, Worst Case and Probabilistic Analysis of the 2-Opt Algorithm for the TSP.
   URL: https://link.springer.com/article/10.1007/s00453-013-9801-4.